

PipeMaster: PonAbe Worked Example

Model Selection, then Parameter Estimation, from RAD-seq Data

Contents

0. Install or update PipeMaster	1
1. Setup	2
1.1 The data	2
1.2 The model, attached to the data	2
1.3 Observed summary statistics	3
2. Part I — Which model?	3
2.1 The competing model	4
2.2 Reference tables	4
2.3 Can the observation be produced by either model?	5
2.4 First, the simple way: model choice by pure rejection	8
2.5 Train the classifier	9
2.6 Classify the observation	10
2.7 Is the observation inside the class it was assigned?	10
3. Part II — Parameter estimation under the selected model	12
3.1 Which parameters are free?	12
3.2 First, the simple way: estimation by pure rejection	12
3.3 Prior-predictive check	14
3.4 Train the regression network	16
3.5 Posterior	16
3.6 Post-training diagnostics	19
4. Simulation budget	21
References	21

This document runs the complete PipeMaster pipeline on the **PonAbe_TwoSpecies** example data shipped with the package — Bornean (*Pongo pygmaeus*) and Sumatran (*Pongo abelii*) orangutans. Nothing is downloaded, and every figure below is produced by the code shown above it.

The order matters. We ask **which model** first, and only then **what its parameters are**. Estimating parameters under a model the data reject gives precise, confident, meaningless numbers.

0. Install or update PipeMaster

Do this first, even if you already have PipeMaster installed. This tutorial relies on recent fixes to `get.data.structure()`, `observed.sumstats()` and `nn.predict.classify()`. On an older installation it does not merely give different numbers – it stops with an error in section 1.2 and again in section 2.6.

```
install.packages("devtools")
devtools::install_github("gehara/PipeMaster")
```

Restart R after installing, then load it:

```
library(PipeMaster)
set.seed(42)

OUT <- "ponabe_output"
dir.create(OUT, showWarnings = FALSE)
```

PipeMaster pulls in `ape`, `e1071`, `msm` and `torch` automatically. `torch` downloads its own C++ backend the first time it is used, which needs a working internet connection and a few minutes – do it before the session rather than during it:

```
torch::torch_is_installed() # should be TRUE
```

Two optional packages are worth having: `RANN` speeds up the nearest-neighbour searches in the OOD diagnostics, and `bigmemory` reduces memory use when training with several cores.

```
install.packages(c("RANN", "bigmemory"))
```

The first code chunk that touches the data will confirm your installation is current: if `get.data.structure()` asks you for a `chrom.sizes` argument, your copy of PipeMaster predates the fix and you need to reinstall.

1. Setup

1.1 The data

Twenty diploid individuals, ten per species, typed at 10,000 RAD loci of 100 bp each.

```
extdir <- system.file("extdata", "PonAbe_TwoSpecies", package = "PipeMaster")
list.files(extdir)

## [1] "model_diffuse.RData"      "model_informative.RData"  "obs_allstats_wgs.txt"
## [4] "PonAbe_TwoSpecies_RAD.vcf" "pop_assign.txt"

vcf_file <- file.path(extdir, "PonAbe_TwoSpecies_RAD.vcf")
pop_assign <- read.table(file.path(extdir, "pop_assign.txt"), header = TRUE)
table(pop_assign$pop)

##
##  1  2
## 10 10
```

1.2 The model, attached to the data

`model_informative.RData` ships configured for whole genomes (10,000 loci x 100 kb). `get.data.structure()` re-shapes it onto the RAD data by reading the `##contig` header of the VCF.

```
load(file.path(extdir, "model_informative.RData"))
model_IM <- PonAbe_IM_WGS_informative
model_IM$labels$name <- "IM"

model_IM <- get.data.structure(model_IM, path.to.vcf = vcf_file,
                              pop.assign = pop_assign)
```

```
## PipeMaster:: 10000 loci declared by the VCF ##contig header
## PipeMaster:: Scanning VCF for chromosomes carrying variants...
## PipeMaster:: 5533 with variants, 4467 invariant (retained)
## PipeMaster:: VCF data structure: 10000 chromosomes, 1000000 total sites, 20/20 haploid samples per p
```

```
cat(sprintf("%d loci x %s bp, %s haplotypes\n",
           nrow(model_IM$loci), model_IM$loci[1, 2],
           paste(model_IM$I[1, 4:5], collapse = " + ")))
```

```
## 10000 loci x 100 bp, 20 + 20 haplotypes
```

What `get.data.structure()` does. A model carries its priors *and* a description of the data it applies to: how many loci, how long each is, and how many haplotypes per population. This rewrites that description (`model$loci` and `model$I`) to match a real dataset, leaving every prior untouched. For VCF input it reads the locus list and lengths from the `##contig=<ID=...,length=...>` header; for PHYLIP it takes them from the alignments. Simulations must have the same locus structure as the data, or the simulated and observed statistics are not comparable.

Of those 10,000 loci, only 5,533 carry a variant. The other 4,467 are **kept**: a RAD locus that came back monomorphic is evidence of low diversity, and it belongs in the per-locus denominator of every summary statistic. Dropping it inflates π , θ_W and F_{st} alike.

1.3 Observed summary statistics

```
obs <- observed.sumstats(model_IM, path.to.vcf = vcf_file,
                        pop.assign = pop_assign, verbose = FALSE)

round(obs[1, c("s_mean_pi", "s_mean_thetaW", "s_mean_Fst", "s_mean_segs")], 5)
```

```
##      s_mean_pi s_mean_thetaW      s_mean_Fst      s_mean_segs
##      0.17305      0.19429      0.07087      0.82640
```

What `observed.sumstats()` does. It converts the VCF to internal haplotype matrices and computes summary statistics with the *same C code the simulator uses*. That shared implementation is the point: observed and simulated statistics must be computed identically, or the network learns the difference between the two pipelines rather than the difference between demographic histories.

It returns two blocks of numbers, 545 in total here:

- **104 summary statistics.** Each statistic is computed per locus, then summarised across loci by its first four moments – hence the `s_mean_`, `s_var_`, `s_skew_` and `s_kurt_` prefixes. The variance and shape terms carry real information: two histories can share a mean π while differing in how variable π is across the genome. Suffixes mark the population: `_1` Bornean, `_2` Sumatran, bare names pooled, `_1_2` pairwise. The underlying quantities are **segs** (segregating sites), **pi** (nucleotide diversity), **thetaW** (Watterson's θ), **tajd** (Tajima's D), **ZnS** (linkage disequilibrium), **Fst**, plus **shared**, **private** and **fixed** differences between the pair.
- **441 folded SFS bins** – the joint site frequency spectrum, 21 x 21 for 20 + 20 haplotypes. Folded because the ancestral allele is unknown.

These are values per locus averaged over all 10,000 loci, which is why the invariant loci in the previous section matter so much: they are 4,467 zeros in the denominator.

2. Part I — Which model?

The biological question is whether Bornean and Sumatran orangutans exchanged genes after they split, or diverged in complete isolation. We build both models and simulate under each, then answer the question twice: first with plain ABC rejection, which has no moving parts, and then with a neural classifier. Doing it in that order shows what the network buys you, and why.

2.1 The competing model

Strict isolation is the IM model with migration deleted. Everything else – priors, divergence time, locus structure – is identical, so the comparison isolates exactly one factor.

```
model_ISO <- model_IM
model_ISO$flags$m <- NULL
model_ISO$labels$name <- "ISO"

# confirm the -m flag really disappeared from the ms command
c(IM = grepl(" -m ", PipeMaster::msABC.commander(model_IM, arg = 1)[[1]]),
  ISO = grepl(" -m ", PipeMaster::msABC.commander(model_ISO, arg = 1)[[1]]))

##      IM      ISO
## TRUE FALSE
```

2.2 Reference tables

What `sim.sumstats()` does. For each simulation it (1) draws one value per free parameter from that parameter’s prior, honouring the model’s conditions, (2) converts them to coalescent units and builds an `ms` command, (3) simulates genealogies and mutations for all 10,000 loci with the built-in `msABC` engine, and (4) computes the same 545 statistics as `observed.sumstats()`. One row of the reference table = one parameter draw plus the statistics it produced.

This is the “reference table” that everything downstream consumes: the classifier learns which model produces which statistics, and the regression network learns which parameter values produce which statistics. Simulation is the only place the demographic model actually enters – every later step sees only this table.

`batch.size` controls how many simulations go to each C call (memory and progress granularity, not the total); `ncores` sets independent worker processes. Use `sim.scrm.sumstats()` instead for long loci with recombination, where the `scrm` engine is far faster.

Equal numbers per model: the classifier learns whatever class balance you give it, so an unbalanced table biases the model probabilities directly.

```
NSIMS <- 2000
CORES <- 8

sim.sumstats(model_IM, nsims = NSIMS, batch.size = 8, mu.rates = 2e-8,
              ncores = CORES, path = OUT, output.name = "IM")

sim.sumstats(model_ISO, nsims = NSIMS, batch.size = 8, mu.rates = 2e-8,
              ncores = CORES, path = OUT, output.name = "ISO")

rt_IM <- read.table(file.path(OUT, "SIMS_IM.txt"), header = TRUE)
rt_ISO <- read.table(file.path(OUT, "SIMS_ISO.txt"), header = TRUE)

param_cols <- c("Ne0.pop1", "Ne0.pop2", "Ne1.pop1", "Ne1.pop2",
               "join1", "mig0.1_2", "mig0.2_1")
derived <- c("Ne.anc_1_2", "t.Ne1.pop1", "t.Ne1.pop2", "t.Ne.anc_1_2")
nuisance <- c("mean.rate", "sd.rate")
stat_cols <- setdiff(colnames(rt_IM), c(param_cols, derived, nuisance))

cat(sprintf("IM  %d x %d\nISO %d x %d\n%d statistics\n",
            nrow(rt_IM), ncol(rt_IM), nrow(rt_ISO), ncol(rt_ISO),
            length(stat_cols)))

## IM  2048 x 558
```

```
## ISO 2048 x 556
## 545 statistics

obs_raw <- setNames(as.numeric(obs[1, stat_cols]), stat_cols)
```

The ISO table has two fewer columns – no mig0.1_2, no mig0.2_1 – so the two are stacked on the statistics they share, plus a label:

```
rt_both <- rbind(
  data.frame(model = "IM", rt_IM[, stat_cols], check.names = FALSE),
  data.frame(model = "ISO", rt_ISO[, stat_cols], check.names = FALSE))

table(rt_both$model)
```

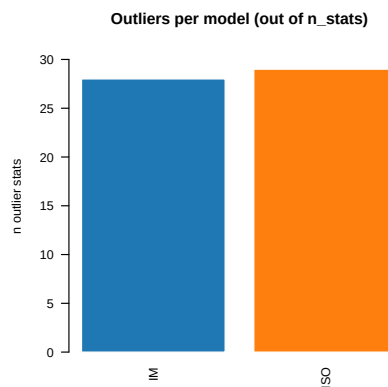
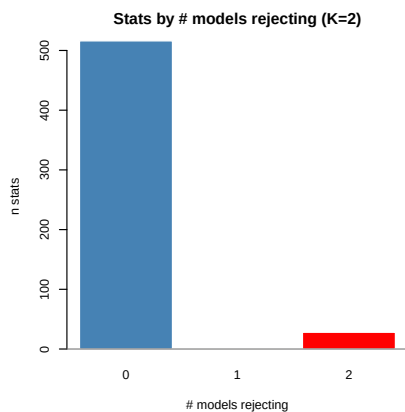
```
##
##   IM   ISO
## 2048 2048
```

2.3 Can the observation be produced by either model?

Run this **before** training. It asks whether the observed data lie inside each model's prior-predictive support.

```
pre_c <- OOD.pretrain.classify(reftable = rt_both, model_col = "model",
                              observed = obs_raw, stat_cols = stat_cols,
                              plot = TRUE, verbose = FALSE)
```

```
## PipeMaster:: pls.fit: 297 stats -> 2 PLS components (54.5% variance)
```



Per-class PCA-NN verdict

PASS=inside, WARN=edge, FAIL=outside the sim cloud

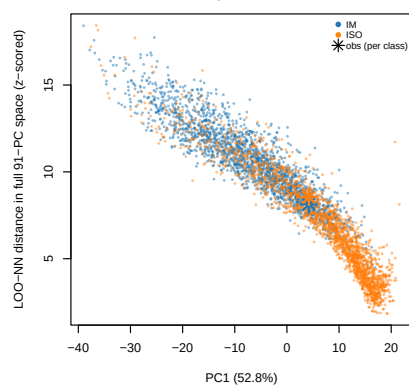
Universal outliers: 28 / 545 (5.1%)

IM : PASS

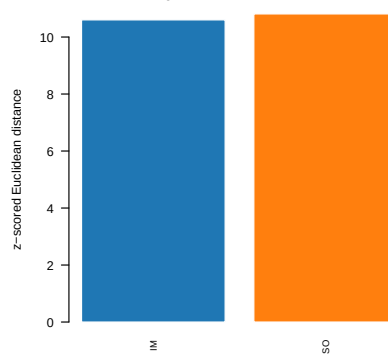
ISO : PASS

Drop 28 universal-outlier stats from features

PCA: PC1 vs LOO-NN (4096 sims, 297 stats, 91 PC:



PCA-space d_obs to nearest sim

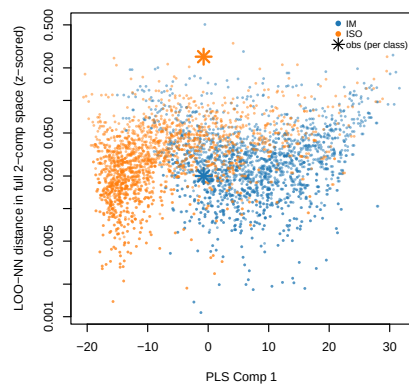


PCA per-class verdict

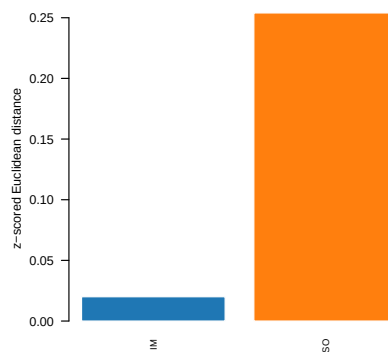
PASS=inside WARN=edge FAIL=outside

class	verdict	pct
IM	PASS	69.3
ISO	PASS	94.6

PLS-DA: Comp1 vs LOO-NN (2 comps, 297 stats)



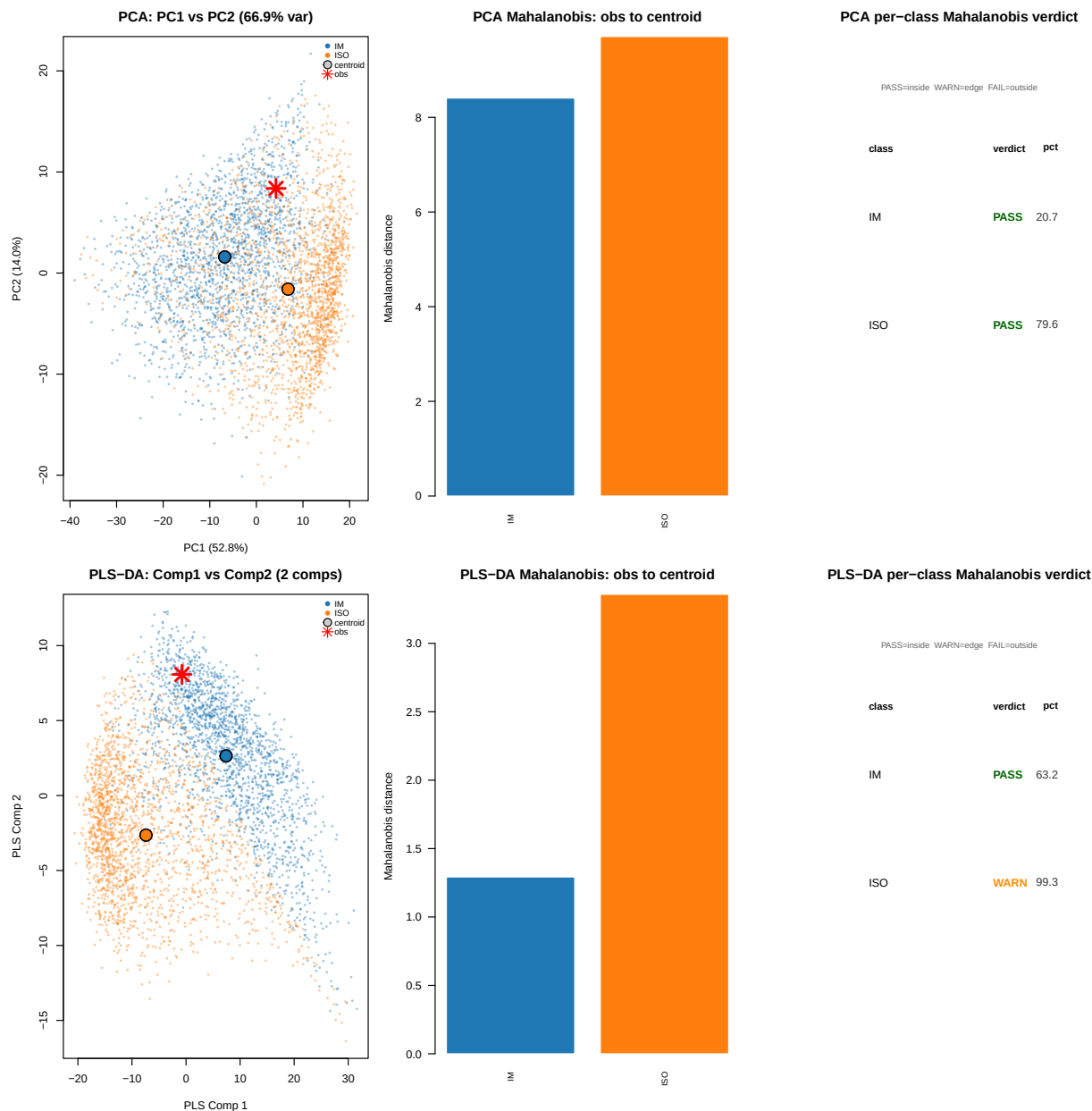
PLS-DA-space d_obs to nearest sim



PLS-DA per-class verdict

PASS=inside WARN=edge FAIL=outside

class	verdict	pct
IM	PASS	38.7
ISO	WARN	100.0



```
cat("verdict:", pre_c$verdict, "\n")
```

```
## verdict: pass
```

```
cat("universal outliers:", sum(pre_c$universal_mask), "of", length(stat_cols), "\n")
```

```
## universal outliers: 28 of 545
```

What `OOD.pretrain.classify()` does. The multi-model counterpart of `OOD.pretrain()`. It runs the per-statistic support check **separately for each candidate model**, then projects every model's simulations plus the observation into one shared PCA and one shared **PLS-DA** (discriminant analysis: components chosen to separate the classes, rather than to track parameters as in the single-model case). The scatter plots show whether the two models occupy distinct regions at all, and where the observation falls relative to both.

The critical output is `universal_mask` – statistics whose observed value falls outside *every* candidate model's

support. No model explains them, so they cannot discriminate between models and are noise to the classifier. Statistics rejected by only *some* models are the informative ones. A large universal fraction fails the check, and the honest response is a better model, not a better classifier.

2.4 First, the simple way: model choice by pure rejection

Before reaching for a neural network, do it the plain way. ABC rejection has no training, no hyperparameters and nothing to go wrong: measure how far each simulation is from the observation, keep the closest few, and count how many came from each model.

`abc.rejection()` does the work. It is internal to the package, so it needs the `:::` prefix. It expects parameter columns, so we code the model label as a number and let it ride along: whatever it accepts, the labels come back attached.

```
rt_rej <- rbind(
  data.frame(model = 1, rt_IM[, stat_cols], check.names = FALSE), # 1 = IM
  data.frame(model = 2, rt_ISO[, stat_cols], check.names = FALSE)) # 2 = ISO

ms_rej <- PipeMaster:::abc.rejection(
  reftable = rt_rej,
  observed = obs_raw,
  param.cols = "model",
  tol = 0.01,
  distance = "sd")

## PipeMaster:: abc.rejection: 4,096 samples, tol=0.0100 → keep 40, distance=sd, pls=FALSE
## PipeMaster:: Accepted 40 samples (tol=0.0100)
## PipeMaster:: Distance range: [76.9982, 82.5877], median=81.5042
## PipeMaster:: Point estimate: model=1.4

counts <- table(factor(c("IM", "ISO")[ms_rej$abc[, "model"]],
  levels = c("IM", "ISO")))

counts

##
## IM ISO
## 25 15

round(prop.table(counts), 3) # posterior model probabilities

##
## IM ISO
## 0.625 0.375
```

What `abc.rejection()` does. It scales every statistic so they contribute comparably – by standard deviation for `distance = "sd"`, by median absolute deviation for `"mad"`, or using the full covariance for `"mahalanobis"` – computes the Euclidean distance from each simulation to the observation, and keeps the closest `tol` fraction. Statistics that never vary get a scale of 1 rather than being dropped, so they contribute nothing when the observation matches them. There is no training and nothing to tune beyond `tol`.

With equal simulations per model and no prior preference between them, the **proportion of accepted simulations from each model is its posterior probability**, and their ratio is the **Bayes factor**.

```
cat(sprintf("Bayes factor IM/ISO = %.2f\n", counts[["IM"]] / counts[["ISO"]]))
```

```
## Bayes factor IM/ISO = 1.67
```

Two things are worth pausing on.

The answer is weak. The accepted set is far from one-sided, so the Bayes factor sits near 1 – barely worth mentioning by any of the usual scales. Yet the classifier in the next section separates these same two models almost perfectly. Both are computed from the same simulations, so the difference is not information in the data; it is how the two methods use it.

Look at the distance range printed above. Even the accepted simulations – the closest 1% of 4,096 – sit at distances of roughly 77 to 83, measured in standard deviations of the statistics. The best match is not a good match. Nothing stands out from the crowd, because across 545 statistics every simulation is far from the observation in *some* direction, and Euclidean distance adds all those directions up with equal weight. This is the **curse of dimensionality**, and it is what blunts plain rejection: only a handful of these statistics respond to migration, and the rest bury that signal in noise. “Closest” stops meaning “most similar in the ways that matter”.

The standard remedies are to hand-pick a few informative statistics, or to reduce dimension first with PLS (`pls = TRUE`, which projects onto components chosen for their relationship to the parameters). The neural network is a third answer: rather than being told which statistics matter, it **learns** the weighting from the simulations. That is the entire reason for the next section.

2.5 Train the classifier

```
clf <- tune.nn.classify(rt_both, model_col = "model",
                      max_epochs = 40L, eta = 3L,
                      n_searches = 2L, cores = 2L, gpus = 0L,
                      seed = 42, verbose = FALSE)

save.classifier(clf, file.path(OUT, "clf"))

cat("validation accuracy:", round(clf$metrics$accuracy, 4), "\n")

## validation accuracy: 1

clf$metrics$confusion

##
##           pred:IM pred:ISO
## true:IM      204      0
## true:ISO       0     204
```

What `tune.nn.classify()` does. Same architecture and same Hyperband search as `tune.nn()`, but the target is a class label rather than a parameter vector, so it minimises **cross-entropy** instead of squared error. Features are augmented and z-scored identically. Parameter columns are detected and excluded automatically – the classifier must decide from statistics alone, since the observed data have no parameters attached. The validation split is **stratified by class**, so both models are represented in the held-out set.

Reading the output. `best_val_loss` is validation cross-entropy (lower is better; $\ln 2 \approx 0.69$ is the value of a coin flip for two classes). The confusion matrix has true classes in rows and predicted in columns, so off-diagonal entries are the simulations the classifier got wrong – and their direction tells you *which* model gets mistaken for which.

Report this number. High accuracy means the two models are **distinguishable given this data structure** – a precondition for the comparison meaning anything. If accuracy sat near 0.5, the models would not be identifiable from these statistics and no probability computed afterwards would be worth reporting.

Accuracy of exactly 1.0 deserves a moment’s suspicion rather than satisfaction: with only 2,000 simulations per class, a validation set of ~100 simulations can be separated perfectly by a model that would still fail on fresh data. Treat it as “clearly separable”, not as a precise number.

`save.classifier()` and `load.classifier()` write and rebuild the trained network. Use them rather than

`saveRDS()`: torch objects hold external pointers that do not survive a plain serialisation round-trip, and restoring one gives an `external pointer is not valid` error.

2.6 Classify the observation

Three output rules, answering slightly different questions:

```
res <- lapply(c("softmax", "mahalanobis", "latent_nn"), function(m)
  nn.predict.classify(clf, observed = obs_raw, method = m,
    reftable = rt_both, model_col = "model", verbose = FALSE))

do.call(rbind, lapply(seq_along(res), function(i) data.frame(
  method = c("softmax", "mahalanobis", "latent_nn")[i],
  best    = res[[i]]$best_model,
  P_IM    = round(res[[i]]$prob_mean[["IM"]], 4),
  P_ISO   = round(res[[i]]$prob_mean[["ISO"]], 4))))
```

```
##      method best  P_IM P_ISO
## 1      softmax  IM 0.9963 0.0037
## 2 mahalanobis  IM 1.0000 0.0000
## 3   latent_nn  IM 0.9551 0.0449
```

- **softmax** – the classifier’s native output. Measures which class *direction* the observation aligns with. It always names a winner, even for an observation lying far outside both classes.
- **mahalanobis** – distance to each class centroid under the pooled within-class covariance. Density-aware; the one to trust when the observation may be out of distribution.
- **latent_nn** – distance to the nearest simulation of each class. Purely geometric.

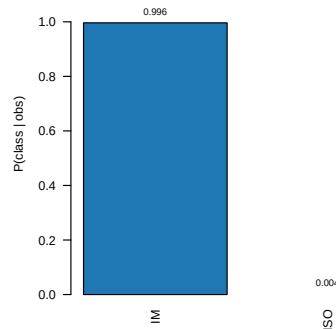
Agreement across all three is the result you want. Disagreement means the observation sits where the classifier is extrapolating.

2.7 Is the observation inside the class it was assigned?

A model probability is only meaningful if the observation actually lies within the winning class’s learned manifold.

```
post_c <- OOD.posttrain.classify(clf, observed = obs_raw, reftable = rt_both,
  model_col = "model", pretrain = pre_c,
  plot = TRUE, verbose = FALSE)
```

Predicted P(class) | margin=0.993



k-NN k=10 (Sun 2022)

PASS=inside WARN=edge FAIL=outside

class	verdict	pct
IM *	PASS	83.3
ISO	FAIL	>100

* = predicted class

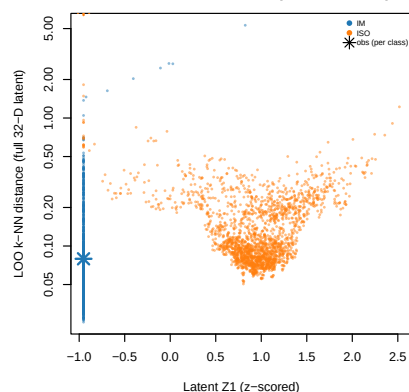
Mahalanobis (Lee 2018)

PASS=inside WARN=edge FAIL=outside

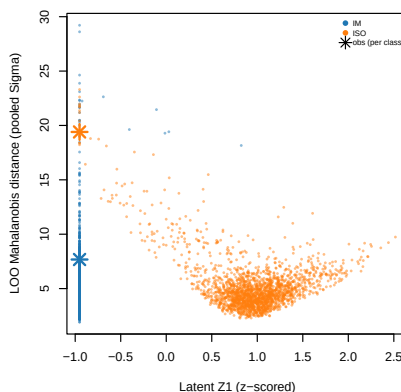
class	verdict	pct
IM *	PASS	92.5
ISO	WARN	99.6

* = predicted class

Latent: Z1 vs LOO k-NN (k=10, 32 dims)



Latent: Z1 vs LOO Mahalanobis



Prediction summary

Predicted class : IM

P(class | obs) : 0.996

Margin (top1-top2) : 0.993

log10 BF (vs runner-up) : 2.43 [decisive]

Bayes factor (pred vs class) :

vs ISO : BF=268.1 log10=2.43

Latent dims : 32

Obs in predicted class : YES

Obs in any class : YES

```
cat("verdict:      ", post_c$verdict, "\n")
```

```
## verdict:      pass
```

```
cat("predicted class:", post_c$predicted_class, "\n")
```

```
## predicted class: IM
```

```
cat("margin:      ", round(post_c$margin, 4), "\n")
```

```
## margin:      0.9926
```

What `OOD.posttrain.classify()` does. It takes the classifier’s penultimate-layer representation – the space in which it separates the models – and asks, for each class, whether the observation lies inside that class’s cloud of simulations, using the same nearest-neighbour test against a leave-one-out null. It then cross-references the answer with the predicted class.

This is the check that makes a model probability meaningful. A softmax always sums to one and will hand you 0.99 for an observation sitting nowhere near either class: the classifier reports which side of its decision boundary you are on, and that boundary extends to infinity. Asking separately whether you are *near any training data* is what distinguishes “the data favour IM” from “IM is the closer of two models that both fail”.

`$margin` is $P(\text{top } 1) - P(\text{top } 2)$, a one-number measure of how decisive the call was. A high margin with a fail verdict is the dangerous combination – confident and unfounded.

Verdict	Meaning
pass	Observation lies inside the predicted class. Probability is usable.

Verdict	Meaning
warn	Inside <i>some</i> class, but not the predicted one. Prediction unreliable.
fail	Outside every class. The classifier is extrapolating; the probability is meaningless.

With real data `fail` is common and important: it means none of your candidate models can produce the observed data, so the “winner” is merely the least-bad of a bad set. Add a model rather than report the ranking. When that happens, `OOD.attribute.classify()` attributes the out-of-distribution-ness to individual statistics via integrated gradients.

3. Part II — Parameter estimation under the selected model

Only now, having established that the data prefer IM *and* that the observation lies inside the IM manifold, do we estimate parameters.

3.1 Which parameters are free?

Four of the eleven parameter columns are not free – they are deterministic consequences of the model’s conditions. Feeding them to the network asks it to learn duplicates.

```
c(times_equal_join = all(rt_IM$t.Ne1.pop1 == rt_IM$join1),
  anc_is_sum       = all(rt_IM$Ne.anc_1_2 == rt_IM$Ne1.pop1 + rt_IM$Ne1.pop2))
```

```
## times_equal_join      anc_is_sum
##                   TRUE          FALSE
```

```
param_cols
```

```
## [1] "Ne0.pop1" "Ne0.pop2" "Ne1.pop1" "Ne1.pop2" "join1"      "mig0.1_2" "mig0.2_1"
```

`t.Ne1.pop1`, `t.Ne1.pop2` and `t.Ne.anc_1_2` all equal `join1` through the model’s = time conditions; `Ne.anc_1_2` is the sum of the daughter sizes because `model$sum_anc_ne` is `TRUE`. Verify this for your own model rather than assuming it – the conditions are model-specific.

3.2 First, the simple way: estimation by pure rejection

Same principle as the model-choice rejection above, but now we keep the *parameter values* of the accepted simulations instead of counting their labels. `abc.rejection()` does this directly. It is internal, so it needs the `:::` prefix.

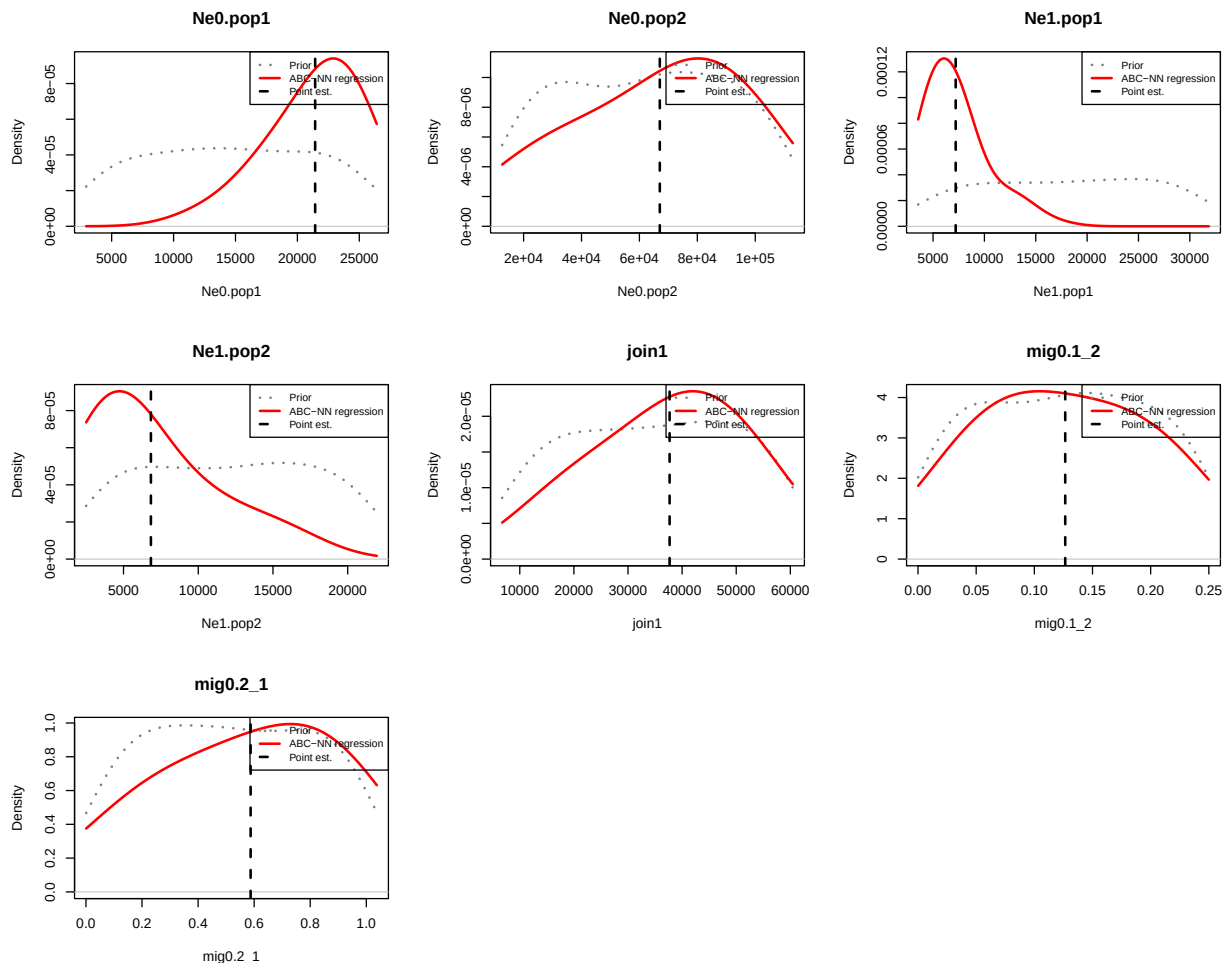
```
post_rej <- PipeMaster:::abc.rejection(
  reftable = rt_IM[, c(param_cols, stat_cols)],
  observed = obs_raw,
  param_cols = param_cols,
  tol = 0.05,
  distance = "sd")
```

```
## PipeMaster:: abc.rejection: 2,048 samples, tol=0.0500 → keep 102, distance=sd, pls=FALSE
## PipeMaster:: Accepted 102 samples (tol=0.0500)
## PipeMaster:: Distance range: [81.8455, 89.9777], median=88.5496
## PipeMaster:: Point estimate: Ne0.pop1=21434.7 Ne0.pop2=67039.5 Ne1.pop1=7203.3 Ne1.pop2=6832.1 join1=
summary(post_rej)
```

```
## Point estimate:
## Ne0.pop1 Ne0.pop2 Ne1.pop1 Ne1.pop2      join1 mig0.1_2 mig0.2_1
## 21434.73 67039.48 7203.28 6832.07 37688.96      0.13      0.59
##
## ABC-NN regression posterior:
##           Mean      Median      Mode      2.5%      25%      50%      75%      97.5%
## Ne0.pop1 21417.17 22430.16 23200.41 12801.87 18986.15 22430.16 23927.20 26102.88
## Ne0.pop2 67014.58 74131.52 85105.74 15566.10 44893.84 74131.52 88225.39 108631.51
## Ne1.pop1 7220.80 6498.52 5787.62 3635.38 4887.44 6498.52 8336.09 14799.40
## Ne1.pop2 6841.60 4774.68 4120.34 2611.53 3759.95 4774.68 9574.83 16414.15
## join1    37716.30 40338.44 42714.26 11826.52 27913.65 40338.44 48414.26 59059.03
## mig0.1_2 0.13     0.12     0.07     0.01     0.07     0.12     0.18     0.24
## mig0.2_1 0.59     0.63     0.85     0.05     0.35     0.63     0.85     1.00
```

What `abc.rejection()` does. Scales each statistic (by its standard deviation for `distance = "sd"`, by median absolute deviation for `"mad"`, or using the full covariance for `"mahalanobis"`), computes Euclidean distance from every simulation to the observation, and keeps the closest `tol` fraction. The accepted rows' parameter values *are* the posterior sample – no adjustment, no model fitting. The point estimate is their distance-weighted mean. Setting `pls = TRUE` projects the statistics onto PLS components first, which is the standard fix for the dimensionality problem seen above.

```
plot(post_rej, bw.adjust = 2)
```



This is the honest baseline, and it has one guarantee the network cannot offer: every value in the posterior is a parameter that was actually simulated, so the result can never leave the prior. Compare these estimates with the network’s in section 3.5 – where the network wanders outside the prior, rejection by construction cannot.

The cost is resolution. With 2048 simulations and `tol = 0.05`, the posterior rests on roughly 100 accepted simulations, and the same dimensionality problem from section 2.4 applies: distances are computed over hundreds of statistics of which only a few are informative. Expect posteriors that look close to the prior for parameters the statistics constrain weakly.

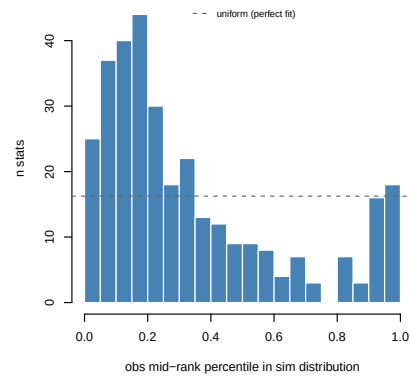
(A cosmetic wrinkle: `summary()` labels this block “ABC-NN regression posterior” because it shares the `posterior` class with `nn.predict()` output. No network was involved here.)

3.3 Prior-predictive check

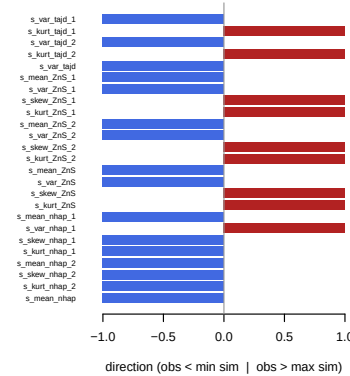
```
pre <- OOD.pretrain(observed = obs_raw,
                    reftable = rt_IM[, c(param_cols, stat_cols)],
                    stat_cols = stat_cols, param_cols = param_cols,
                    plot = TRUE, verbose = FALSE)

## PipeMaster:: pls.fit: 324 stats -> 10 PLS components (80.5% variance)
## PipeMaster:: pls.fit: 297 stats -> 10 PLS components (80.4% variance)
```

istribution (325 informative: 297 ok + 28 outlier; 220 unin



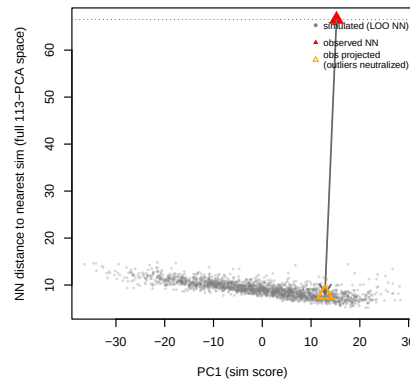
Outliers: 25 of 28 (red: obs>max sim, blue: obs<



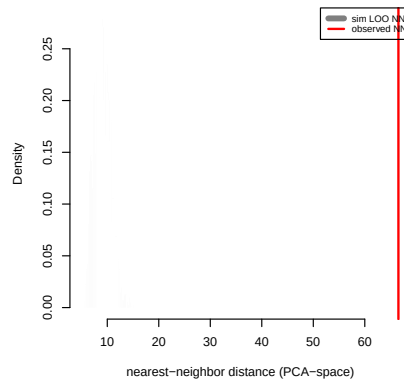
Verdict summary

OVERALL: PASS
 Per-stat: PASS
 28 outliers / 325 informative (8.6%)
 220 uninformative
 545 stats total
 PCA all: d= 66.53 p=0.0000 [FAIL]
 PCA filtered: d= 7.04 p=0.8467 [PASS]
 PLS all: d= 42.35 p=0.0000 [FAIL]
 PLS filtered: d= 3.85 p=0.4771 [PASS]
 Tier rules:
 pass < 10%
 warn 10-25%
 fail > 25%

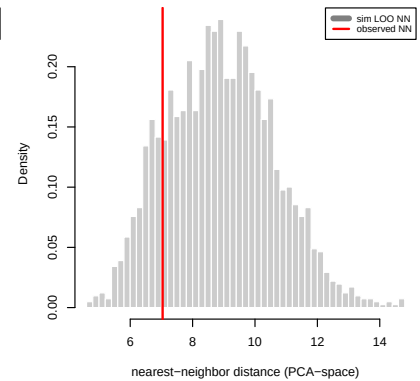
NN vs PCA1 (324 informative of 324 stats, PCA basi



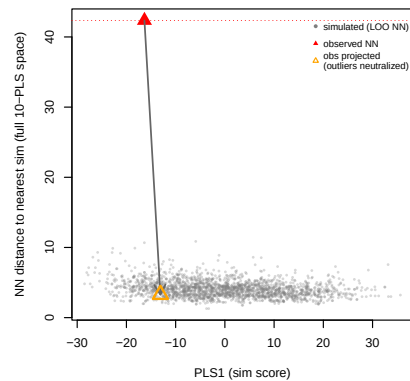
NN distance all (PCA, 113 dims, p=0.000) [FAIL]



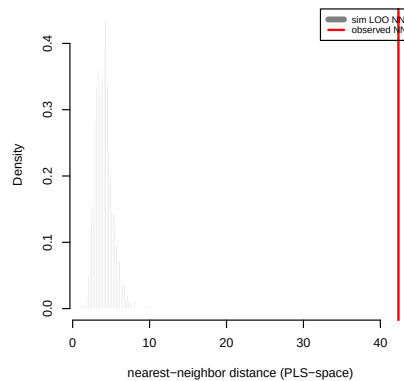
NN distance filtered (PCA, 113 dims, p=0.847) [PASS]



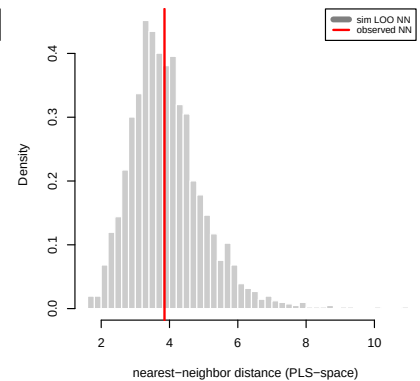
NN vs PLS1 (324 informative of 324 stats, PLS basi



NN distance all (PLS, 10 dims, p=0.000) [FAIL]



NN distance filtered (PLS, 10 dims, p=0.477) [PASS]



```
cat("verdict:", pre$overall, "\n")
```

```
## verdict: pass
```

```
cat("outlier statistics:", sum(pre$percentiles$outlier),  
    "of", length(stat_cols), "\n")
```

```
## outlier statistics: 28 of 545
```

What `OOD.pretrain()` does. Three independent checks that the observed data are reachable under this model and prior, all run *before* training so you can stop early:

1. **Per-statistic support.** For each of the 545 statistics, does the observed value fall inside the range the simulations produced? The fraction outside sets a tier: pass below 10%, warn to 25%, fail above.
2. **Nearest-neighbour density in PCA space.** Statistics are correlated, so an observation can sit inside every marginal range yet in a hole in the joint distribution. This projects to principal components

and compares the distance from the observation to its nearest simulation against the distribution of simulation-to-simulation distances. A p-value near zero means the observation is isolated.

3. **The same test in PLS space**, using components chosen to correlate with the parameters rather than to maximise variance. Passing `param.cols` enables it. PLS can expose a problem PCA misses when the parameter-relevant signal is a small share of total variance.

Each runs on all statistics and again on the subset that survived check 1, because a handful of wild outliers can dominate a distance in 545 dimensions.

Reading the output. `$overall` combines the tiers. `$percentiles` holds each statistic's position within the simulated distribution, and `$percentiles$outlier` flags those outside – inspect them with `OOD.outliers()`, and drill into the projections with `OOD.projection.diagnose()`.

Note the accessor: `OOD.pretrain()` and `OOD.posttrain()` report in `$overall`, while the classifier variants above use `$verdict`.

3.4 Train the regression network

```
tune_result <- tune.nn(rt_IM, param.cols = param_cols, type = "sumstat",
  exclude.cols = c(derived, nuisance),
  max_epochs = 40L, eta = 3L,
  n_searches = 2L, top_k = 2L,
  cores = 2L, gpus = 0L, seed = 42, verbose = FALSE)

save.tune.result(tune_result, file.path(OUT, "tune_IM"))

cat("best validation loss:", round(tune_result$best_val_loss, 5), "\n")
```

```
## best validation loss: 0.0231
```

What `tune.nn()` does. It learns the inverse map, statistics $\rightarrow$ parameters, by regression on the reference table. Three details matter for reading the output:

- **Feature augmentation.** Each statistic enters twice, as `cbind(raw, log1p(abs(raw)))`, then z-scored. Our 545 statistics become 1090 inputs. The log copy lets the network use both absolute and relative changes in quantities like θ_W that vary over orders of magnitude.
- **Targets are modelled in log space**, z-scored, and inverted with `exp()`. This keeps N_e and time parameters positive and makes their priors roughly uniform on the learned scale. **It also means predictions are not confined to the prior range** – `exp()` of an extrapolated z-score can land anywhere. An estimate outside the prior is therefore a symptom of an under-trained or extrapolating network, not a bug.
- **Hyperband search.** `n_searches` independent searches each run a full Hyperband: many configurations get a small training budget, the worst are discarded, survivors get `eta`-times more, repeat. `top_k` keeps the best models as an ensemble rather than one winner.

`best_val_loss` is mean squared error on the held-out split, in z-scored log units – useful for comparing runs on the same data, not interpretable as a parameter error. `r2` and mean percentage error in original units are in `tune_result$metrics`.

Ensemble prediction. The point estimate is not a plain average of the `top_k` models. Validation simulations are weighted by a Gaussian kernel on their distance to the observation, each model's validation loss is recomputed under those weights, and the models are combined by a softmax of those *local* losses. A model that is mediocre overall but accurate in the neighbourhood of your data is weighted up.

3.5 Posterior

Two uncertainty methods, one call each, merged for plotting.


```

post_boot <- nn.predict(tune_result, observed = obs_raw, reftable = rt_IM,
                        param.cols = param_cols, method = "bootstrap",
                        n_boot = 2000L, seed = 42, verbose = FALSE)

post_abc <- nn.predict(tune_result, observed = obs_raw, reftable = rt_IM,
                       param.cols = param_cols, method = "ABC_NN_regression",
                       tolerance = 0.2, seed = 42, verbose = FALSE)

pred <- post_abc
pred$bootstrap <- post_boot$bootstrap
class(pred) <- "posterior"

summary(pred)

## Point estimate:
## Ne0.pop1 Ne0.pop2 Ne1.pop1 Ne1.pop2      join1 mig0.1_2 mig0.2_1
## 47944.36 26530.46 3990.38   784.57 89394.42      0.29    0.54
##
## Residual bootstrap (prediction intervals):
##           Mean   Median     Mode    2.5%    25%    50%    75%    97.5%
## Ne0.pop1 26412.17 26412.17 26381.65 26412.17 26412.17 26412.17 26412.17 26412.17
## Ne0.pop2 29025.34 27276.76 26463.06 16824.91 25030.00 27276.76 32551.79 47798.57
## Ne1.pop1  4675.09  4334.12  3591.48  3552.13  3552.13  4334.12  5221.67  7725.17
## Ne1.pop2  2777.78  2500.50  2503.55  2500.50  2500.50  2500.50  2500.50  5380.65
## join1    60452.92 60452.92 60383.07 60452.92 60452.92 60452.92 60452.92 60452.92
## mig0.1_2    0.25    0.25    0.25    0.24    0.25    0.25    0.25    0.25
## mig0.2_1    0.55    0.55    0.54    0.36    0.52    0.55    0.60    0.75
##
## ABC-NN regression posterior:
##           Mean   Median     Mode    2.5%    25%    50%    75%    97.5%
## Ne0.pop1 26412.17 26412.17 26370.25 26412.17 26412.17 26412.17 26412.17 26412.17
## Ne0.pop2 24587.86 24252.06 23330.15 16238.50 21566.88 24252.06 27491.13 34250.99
## Ne1.pop1  3576.44  3552.13  3552.38  3552.13  3552.13  3552.13  3552.13  3829.90
## Ne1.pop2  2520.69  2500.50  2500.62  2500.50  2500.50  2500.50  2500.50  2500.50
## join1    60452.92 60452.92 60356.98 60452.92 60452.92 60452.92 60452.92 60452.92
## mig0.1_2    0.25    0.25    0.25    0.25    0.25    0.25    0.25    0.25
## mig0.2_1    0.53    0.52    0.51    0.43    0.50    0.52    0.56    0.66

```

Reading summary(). *Point estimate* is the proximity-weighted ensemble prediction described above – a single best guess, no uncertainty attached. *Residual bootstrap* gives the median and quantiles of the bootstrap replicates. *ABC* gives the same for the regression-adjusted accepted simulations. The two intervals are not interchangeable: compare the bootstrap against itself across runs to judge network stability, and report the ABC interval as the posterior.

Both are back-transformed to natural units – Ne in individuals, join1 in generations, migration in Nm (migrants per generation) since `model$mig_scale` is "Nm".

These two are not two ways of computing the same thing. They answer different questions, and it is worth being precise about how each works.

What the residual bootstrap does. It asks: *how wrong is this network usually, on simulations that look like my data?*

1. Run the trained network on the **validation** simulations – data it never trained on – and compute residuals, `true - predicted`, on the original parameter scale.
2. Measure the Euclidean distance from every validation simulation to the observed data in standardised

statistic space.

3. Weight each residual by that distance through an **Epanechnikov kernel**, $w \propto 1 - u^2$ for $u = d/h \leq 1$, with the bandwidth h set to the 10th percentile of distances. Residuals from simulations unlike your data get zero weight. (If fewer than 50 survive, it falls back to a Gaussian kernel with $h = \text{median distance}$, so the interval never collapses.)
4. Draw `n_boot` residuals with replacement in proportion to those weights and add each to the point estimate.

So the interval is *centred on the point estimate* and its width is the local error of the network. The prior never enters. This is a **prediction interval**: it tells you how much the network's answer would wobble, not what the data say about the parameter. It can sit entirely outside the prior.

What ABC-NN regression does. It asks: *which parameter values could have generated data like mine?* This is Beaumont et al. (2002), with the network serving as the regression function.

1. Compute $\|S_i - S_{obs}\|$ for **every row of the reference table** in standardised statistic space.
2. Accept the closest **tolerance** fraction – plain ABC rejection so far.
3. The accepted rows carry their own true parameter values θ_i , drawn from the prior. A plain ABC posterior would stop here and report them.
4. **Regression adjustment**: each accepted θ_i is shifted by how the network's prediction differs between that simulation and the observation,

$$\theta_i^{adj} = \theta_i - (g(S_i) - g(S_{obs}))$$

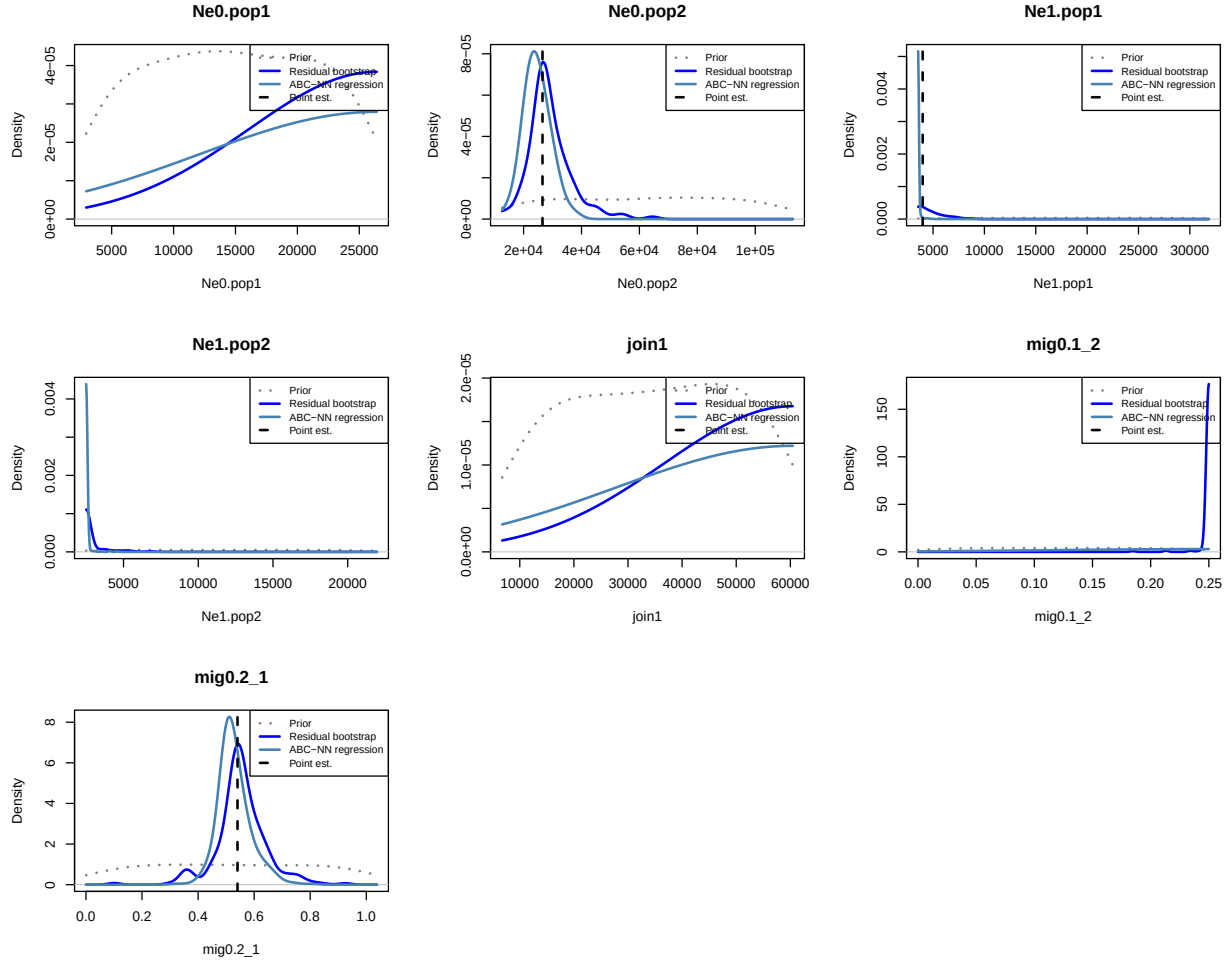
correcting for the fact that an accepted simulation is *near* the observation but not identical to it.

Because it starts from parameter values that were actually drawn from the prior and only nudges them, the result is a **posterior**: prior-supported by construction, and it cannot leave the prior range by more than the adjustment.

The practical consequence: if the two disagree sharply, trust the ABC posterior about the parameter and read the bootstrap as a statement about how much the network is struggling.

`tolerance = 0.2` keeps 20% of simulations, appropriate at this small budget. At production scale (50,000+ simulations) drop it to 0.05.

```
plot(pred, bw.adjust = 2)
```



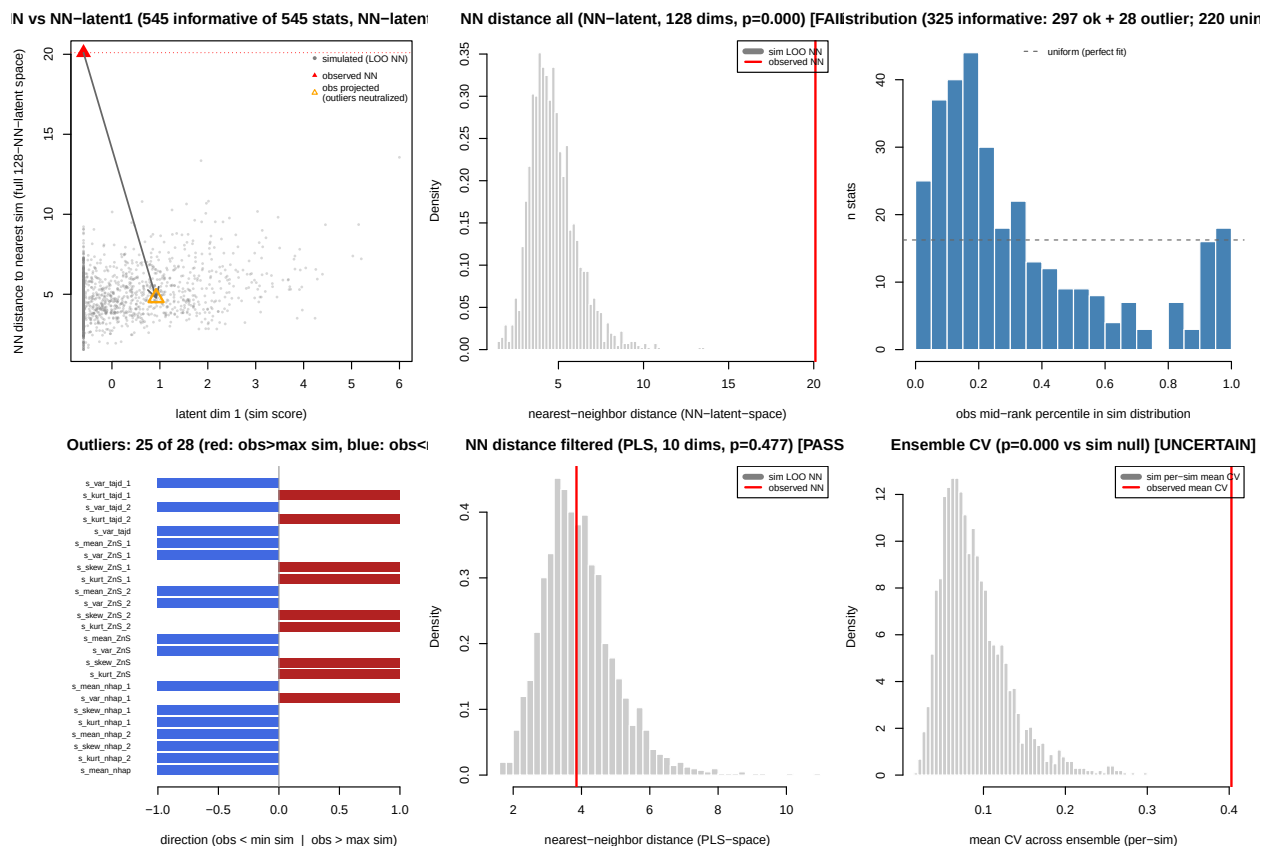
One panel per free parameter. Each shows the **prior** (grey), the **residual bootstrap** (blue) and the **ABC posterior** (steel blue), so the question “did the data actually tell us anything?” is answered by eye: a posterior that simply reproduces the prior means the statistics carry no information about that parameter, and one pinned against a prior edge means the bound is doing the work and should be widened. `bw.adjust` only smooths the density estimate; it does not change the samples.

3.6 Post-training diagnostics

```
# note: OOD.posttrain() has no `verbose` argument, unlike OOD.pretrain()
post <- OOD.posttrain(trained.nn = tune_result, observed = obs_raw,
  reftable = rt_IM[, c(param_cols, stat_cols)],
  pretrain = pre, plot = TRUE)
```

```
## PipeMaster:: OOD.posttrain (sumstat)
## Stats: 545 total = 325 informative (297 ok + 28 outlier) + 220 uninformative
## Per-stat tier: outlier_frac = 28/325 = 8.6% [PASS]
## (28 outliers -- inspect with OOD.outliers())
## NN density (PCA all , 324 stats, 113 PCs, 95% var): d_obs=66.528, p=0.0000 [FAIL]
## NN density (PCA filtered, 297 stats, 114 PCs, 95% var): d_obs=7.039, p=0.8467 [PASS]
## NN density (PLS all , 324 stats, 10 comps): d_obs=42.347, p=0.0000 [FAIL]
## NN density (PLS filtered, 297 stats, 10 comps): d_obs=3.853, p=0.4771 [PASS]
## NN density (NN-latent , 545 stats, 128 dims): d_obs=20.098, p=0.0000 [FAIL]
```

```
## Ensemble disagreement: mean CV=0.403, p=0.0000 vs sim null (2048 sims) [UNCERTAIN]
## Overall: WARN
```



```
cat("verdict:", post$overall, "\n")
```

```
## verdict: warn
```

What `OOD.posttrain()` does. It re-runs the pre-training checks (reusing their PCA and PLS fits when you pass `pretrain =`, which is faster and gives one combined verdict) and adds two that require a *trained* network:

4. **NN-latent density.** The same nearest-neighbour test, but in the penultimate layer's activation space – the representation the network actually regresses from. PCA and PLS are linear projections; the network's internal space is curved, so an observation can look comfortable in every linear view and still sit in a hole in the space where prediction happens. This is the check that most often catches a genuine problem.
5. **Ensemble disagreement.** The `top_k` models all fit the training data well, so where they still disagree is where the data did not constrain them. This computes the coefficient of variation across the ensemble at the observed point and compares it to the same quantity across all simulations. High disagreement means the network is extrapolating exactly where you are asking it to predict – a direct warning that the point estimate is unreliable.

The distinction from `OOD.pretrain()` is worth keeping straight: pre-training asks whether the **model and prior** can produce your data; post-training asks whether the **trained network** can be trusted at your data. A model can pass the first and fail the second simply because the reference table was too small – which is what happens at this document's budget.

4. Simulation budget

Everything above runs at **2,000 simulations per model** so the document knits in minutes. That is a demonstration budget, not an analysis budget, and the diagnostics will say so – expect `OOD.posttrain()` to return **warn** with high ensemble disagreement at this size. That is the tool working correctly.

Measured throughput for this model (10,000 loci x 100 bp, 40 haplotypes): approximately **30,600 simulations/hour on 4 cores** and **73,300/hour on 8**.

Tier	sims / model	4 cores	8 cores	Use for
Demonstration	2,000	~4 min	~2 min	This document
Teaching	10,000	~20 min	~8 min	An interpretable posterior
Production	50,000	~1.7 h	~40 min	Results you would report

For a 7-parameter model, 50,000 simulations is the point at which the posterior stops moving when you add more.

References

- Beaumont M.A., Zhang W. & Balding D.J. (2002) Approximate Bayesian computation in population genetics. *Genetics* 162: 2025-2035.
- Locke D.P. et al. (2011) Comparative and demographic analysis of orang-utan genomes. *Nature* 469: 529-533.
- Lee K. et al. (2018) A simple unified framework for detecting out-of-distribution samples and adversarial attacks. *NeurIPS*.
- Pavlidis P., Laurent S. & Stephan W. (2010) msABC: a modification of Hudson’s ms to facilitate multi-locus ABC analysis. *Molecular Ecology Resources* 10: 723-727.